

Maintenance Programming

Progress Report 1 - Group 4

Contents

- 1 Progress so far
 - 1.1 Forking request
 - 1.2 Running request
 - 1.3 Investigating the test suite
 - 1.4 Investigating HTTPS certificate failures
 - 1.5 Investigating the request implementation
- 2 Outlook
 - 2.1 Planned maintenance types
 - 2.2 Next steps

1 Progress so far

1.1 Forking request

For this course, we plan to work on a fork of request, a legacy Node.js HTTP client library. The original project was officially deprecated in February 2020, and its master branch was last committed to in February 2020. We created a fork and cloned it locally so that the group can maintain the codebase without changing the original repository.

Before making maintenance changes, we created a separate investigation branch. No source-code modifications have been made so far. This gives us a stable baseline against which later maintenance changes can be evaluated.

```
Repository: request
Working branch: prodip-investigation
Environment: Windows / PowerShell
Node.js: 22.19.0
npm: 10.9.3
```

Listing 1: Initial project and environment information.

1.2 Running request

When trying to install the dependencies, the original installation command did not complete successfully. Running `npm install` resulted in an ERESOLVE dependency-tree error involving `karma@3.1.4` and `karma-tap@3.2.1`. The peer-dependency requirement of the latter refers to older Karma versions.

To establish a runnable environment without changing the dependency declarations, we used `npm install --legacy-peer-deps`. This installed 892 packages successfully. The installation reported a large number of deprecated dependencies and 59 security vulnerabilities. We have not used `npm audit fix` because automatically changing the dependency tree would make it harder to distinguish legacy-system problems from changes introduced by dependency upgrades.

The project could then be tested. The lint stage of the `npm test` passed, but `test-ci` failed. Since `npm test` executes the browser tests only after `test-ci` succeeds, the browser stage was not reached.

```
npm test
├─ npm run lint    -> passed
├─ npm run test-ci -> failed
└─ npm run test-browser -> not reached
```

Listing 2: Initial test execution.

The project contains 55 JavaScript test files matching the `tests/test-*.js` pattern. The test runner used by `test-ci` is Taper 0.5.0. Its output was not specific enough to identify the individual failing test, so we ran the test runner independently and then executed relevant tests directly.

1.3 Investigating the test suite

The `test-ci` command uses Taper to execute the JavaScript test files and collect their TAP results. Running `test-ci` separately reproduced the same aggregate failure. This indicated that the next step should be to investigate individual tests rather than immediately modifying the test runner.

One of the relevant tests was `tests/test-https.js`, which exercises both relaxed and strict HTTPS connections. Running this test directly produced 20 assertions: 10 passed and 10 failed.

```
TAP version 13
# tests 20
# pass 10
# fail 10
```

Listing 3: Direct execution of the HTTPS test.

The relaxed HTTPS tests all passed. The strict HTTPS tests all failed with the same TLS error: `CERT_HAS_EXPIRED`. The difference between the two groups made certificate validation the main focus of the following investigation.

1.4 Investigating HTTPS certificate failures

The strict tests configure certificate validation explicitly. They set `strictSSL` to true and provide the CA certificate stored in `tests/ssl/ca/ca.crt`. The relaxed tests instead disable certificate verification using `rejectUnauthorized = false`.

We inspected the certificates used by the HTTPS test server and by the client trust configuration. The server certificate, `tests/ssl/ca/server.crt`, is valid until November 19, 2028. In contrast, the CA certificate, `tests/ssl/ca/ca.crt`, expired on February 27, 2022.

```
server.crt
notBefore = Nov 22 15:23:20 2018 GMT
notAfter  = Nov 19 15:23:20 2028 GMT

ca.crt
notBefore = Mar 1 22:50:56 2012 GMT
notAfter  = Feb 27 22:50:56 2022 GMT
```

Listing 4: Certificate validity periods.

We also checked the Git history of the CA certificate. The file was introduced by commit `37446f5` on March 1, 2012 with the message 'Add a test of HTTPS strict with CA checking'. The `--follow` history contains no later modification of this certificate. Thus, the strict HTTPS test still relies on certificate infrastructure that has remained unchanged since 2012.

To independently verify the certificate chain, we used OpenSSL with the repository CA as the trust anchor. OpenSSL reported that the certificate had expired and that verification failed. We also inspected the server certificate's issuer and subject. Its issuer is `requestCA`, while its subject is `testing.request.mikealrogers.com`. This confirms that the server certificate is issued by the CA used in the strict test.

```
openssl verify -CAfile tests\ssl\ca\ca.crt tests\ssl\ca\server.crt

error 10 at 1 depth lookup: certificate has expired
error tests\ssl\ca\server.crt: verification failed
```

Listing 5: Independent OpenSSL verification.

The evidence therefore indicates that the immediate failure of the strict HTTPS tests is caused by the expired CA certificate in the legacy test infrastructure. The server certificate itself has not expired.

1.5 Investigating the request implementation

After identifying the certificate problem, we inspected `request.js` to determine whether the request library was incorrectly handling the TLS options. The investigation showed that the CA certificate is copied into the HTTPS agent options and that the `rejectUnauthorized` setting is also propagated.

```
if (self.ca) {  
  options.ca = self.ca  
}  
  
if (typeof self.rejectUnauthorized !== 'undefined') {  
  options.rejectUnauthorized = self.rejectUnauthorized  
}
```

Listing 6: Propagation of CA and certificate-verification options.

The code also determines whether the destination or proxy uses HTTPS and includes the relevant TLS options when constructing the connection. Finally, the prepared request options are passed to Node's HTTP/HTTPS implementation through `self.httpModule.request(reqOptions)`.

```
self.req = self.httpModule.request(reqOptions)
```

Listing 7: Delegation to Node's HTTP/HTTPS request implementation.

At the current stage, we therefore have no evidence of a defect in the CA or TLS-option propagation in `request.js`. Instead, the failure is localized to the obsolete certificate used by the strict HTTPS test infrastructure.

2 Outlook

We have now established a reproducible baseline and identified a concrete failure in the legacy HTTPS test infrastructure. The next maintenance step is to repair the certificate setup while preserving the purpose of the strict HTTPS tests. In particular, simply disabling certificate verification would make the tests pass but would remove the behavior that the strict tests are intended to verify.

The solution should therefore be as small as possible and should preserve the original relationship between the test CA and the server certificate. After implementing the change, we will rerun the HTTPS tests and then the complete test suite to determine whether further independent problems remain.

2.1 Planned maintenance types

Based on the work performed so far and the maintenance plan from the initial report, we currently expect the project to involve the following maintenance types. The final classification will be based on the changes that are actually implemented.

- **Corrective maintenance:** restore the intended behavior of the strict HTTPS tests by addressing the invalid certificate chain.
- **Adaptive maintenance:** adapt obsolete HTTPS test infrastructure to the modern execution environment while preserving strict certificate validation.
- **Perfective maintenance:** if the investigation leads to a concrete improvement in the maintainability or clarity of the test infrastructure, such a change may also be included and documented as perfective maintenance.

At this point, these classifications are preliminary. We do not want to classify a maintenance activity as perfective or adaptive until the actual change and its effect on the system have been demonstrated.

2.2 Next steps

- Investigate how the original test certificates were generated and identify the least invasive way to update the expired certificate infrastructure.
- Implement the selected certificate/test-infrastructure change without disabling strict certificate validation.
- Run tests/test-https.js again and verify that both relaxed and strict HTTPS tests behave as intended.
- Run the complete test suite and investigate any remaining failures separately.
- Verify that the maintenance change does not introduce regressions.
- Document the implemented changes and finalize their maintenance classifications.

The work completed so far has therefore moved the project from an undiagnosed baseline failure to a specific, reproducible root-cause finding. The next phase will focus on implementing and validating the corresponding maintenance change.